

RH850/D1L

FCL/Boot 配置

概述

本资料用于帮助使用 RH850/D1L 进行应用程序设计的客户更方便的了解 RH850/D1L 功能模块，其中的代码只是示例程序，并不是以量产为目的设计的，因此，客户在应用过程中，需根据实际应用需求进行程序设计。

1. 目标模块

本资料是主要对 boot 及 FCL 应用的配置进行说明。

1.1 环境

硬件

目标芯片 : RH850/D1L(R7F701403 : 176pin)
: Code Flash ROM: 4M
: Data Flash ROM: 64K
: RAM : 512kb(Local RAM)+16kb(Retention RAM)+144kb(Video RAM)

软件

开发环境 : GHS MULTI 6.1.4

1.2 版本修订历史

版本号	修订者	评审者	修订原因	日期
V1.0	刘凤良	初次做成		2017-1-20

2. 示例程序

示例程序，boot 及 app 代码的配置，基本的设置同之前的说明，相同部分这里不再赘述。

参数设定

参数	设置值
Boot 起始地址	0x00000000
App 起始地址	0x00010000

段的定义和使用

首先，在工程的\device\dr7f701403.ld 文件中定义和配置所需段，关于 FCL 的段定义需在 boot 和 App 两个 ld 文件中同时定义，如下：

Boot 区域：

Boot 区域起始地址设置为 0x00000000，长度自定，需根据芯片 ROM 大小决定。

```
MEMORY
{
  iROM_0      : ORIGIN = 0x00000000, LENGTH = 32k //define boot area
  iROM_1      : ORIGIN = 0x00300000, LENGTH = 32K //ROM for FCL,it's unnecessary
  iROM_1_TOP   : ORIGIN = ., LENGTH = 0
  iRAM_0      : ORIGIN = 0xFEB80000, LENGTH = 512k
  iRAM_1      : ORIGIN = 0xFED80000, LENGTH = 512K
  iRAM_R      : ORIGIN = 0x3FCE4000, LENGTH = 16k
}
```

```
SECTIONS
{
  /* Start of internal ROM area (iROM_0) */

  .intvect                :>iROM_0 /* start of interrupt vector */
  .intvect_end 0x0600     :>. /* end of interrupt vector */
  .rodata                 :>. /* constant datas in ZDA area */
  .robse align(4)         :>. /* initialize textpointer TP for SDA addressing */
}
```

```
...
.rodata ROM(.sldata)      :>. /* constant data to initialize variables in LSDA area (copied to GRAM at
startup)*/

/* FCL Program code */
.R_FCL_CODE_ROM align(4)  :>. /* FCL code in ROM */
.R_FCL_CONST align(4)     :>. /* FCL constants in ROM */
.R_FCL_CODE_USRINT align(4) :>. /* Interrupt code located in RAM */
.R_FCL_CODE_USR align(4)  :>. /* User code located in RAM */
.R_FCL_CODE_RAM align(4)  :>. /* FCL code in RAM */
.R_FCL_CODE_ROMRAM align(4) :>. /* FCL code located in ROM or RAM (depends on the used mode) */
.R_FCL_CODE_RAM_EX_PROT align(4) :>. pad(0x30) align(4):>.

.bcode align(4)           :>iROM_1 /* user defined segment for executable code located in user boot
...

```

```

.FCL_RESERVED align(4) :>iRAM_1 /* FCL copy location */
.data :>. /* initialized data */
.sldata align(4) :>. /* user defined segment for initialized data */
.slbss align(4) :>. /* user defined segment for initialized data */
.bss align(4) :>. /* zero initialized data*/

.sda_start align(4) :>. /* initialize globalpointer GP for SDA addressing */
.sdata align(4) :>. /* initialized data in SDA area*/
.sbss align(4) :>. /* zero initialized data in SDA area*/
.sda_end align(4) :>. /* end of SDA addressing */
.zdata align(4) :>. /* initialized data in ZDA area*/
.zbss align(4) :>. /* zero initialized data in ZDA area*/

.tdata align(4) MAX_SIZE(0x0100) :>. /* initialized and zero-initialized data in TDA area */
.stack align(4) pad(0x2800) :>. /* definition of stack size */

/* Data Section for FCL */
.R_FCL_DATA :>. /* FCL data */
    heapbase align(4) :>

```

App 区域:

注意起始地址偏移，长度根据芯片 ROM 大小设定，设定值必须小于 ROM 大小，本例中 D1L 有 4M ROM，实际设置的只有 (32+3008+32) k。

```

MEMORY
{
    iROM_0 : ORIGIN = 0x00010000, LENGTH = 3008k //define app ROM area
    iROM_1 : ORIGIN = 0x00300000, LENGTH = 32K //ROM for FCL,it's unnecessary
    iROM_1_TOP : ORIGIN = ., LENGTH = 0
    iRAM_0 : ORIGIN = 0xFEB80000, LENGTH = 512k //Local RAM
    iRAM_1 : ORIGIN = 0xFED80000, LENGTH = 512K //Local RAM
    iRAM_R : ORIGIN = 0x3FCE4000, LENGTH = 16k //Retention RAM
}

```

注意中断向量表尾部地址需要根据起始地址进行偏移设置。

```

SECTIONS
{
    /* Start of internal ROM area (iROM_0) */

    .intvect :>iROM_0 /* start of interrupt vector */
    .intvect_end 0x10600 :>. /* end of interrupt vector */
    .rozdata :>. /* constant datas in ZDA area */
    .robss align(4) :>. /* initialize textpointer TP for SDA addressing */
}

```

Note: 在应用时注意做中断向量表地址映射，即代码中 SET_EBASE() 和 SET_PSW_EBV() 函数

FCL:

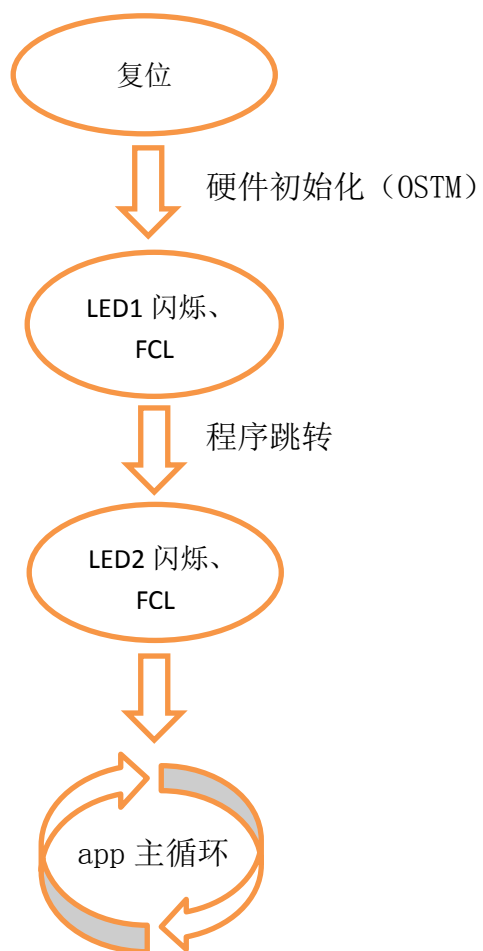
关于 FCL 的详细介绍，请参考 FCL 库相关文档。

如果在 FCL 想擦除全部 ROM，即通过 FCL 同时更新 boot 和 app，需要将 FCL 用到的中断 copy 到 RAM 中，则需要新定义一个中断向量表，请参考本例中的 dr7f701403_startup_ram.850 文件。

2.1 动作概要

上电复位，boot 代码运行，闪灯一段时间后跳转至 app 代码，另一 LED 灯开始闪烁。

动作概要



Boot 程序流程



App 程序流程

